

To-Do App

Complete Documentation

A Comprehensive Guide to Building a Task Management Application with Java, Swing, and MongoDB

Generated on May 11, 2026

Table of Contents

Chapter 1: Main Application Window

Chapter 2: Task List Display

Chapter 3: Task Action Bar

Chapter 4: Database Manager

Chapter 5: Application Bootstrap

Chapter 1: Main Application Window

Welcome to the To-Do App!

Every great journey starts with a first step, and for our application, that step is understanding the "Main Application Window." This is the central control panel or dashboard of our To-Do App, where you'll see everything related to your tasks.

What is the Main Application Window?

Think of the Main Application Window as the central control panel or dashboard of our To-Do App. Just like a car's dashboard shows you your speed, fuel, and warning lights all in one place, our Main Application Window is where you'll see everything related to your tasks. It's the primary screen that pops up when you open the app, and it's where you'll spend most of your time adding, viewing, and managing your to-do items.

The main problem this concept solves is **organization**. Without a main window, all the different parts of our app would be scattered and hard to use. The Main Application Window brings everything together into one coherent, easy-to-understand display.

What You See and Do

When you launch the To-Do App, the first thing you'll encounter is the Main Application Window. From this window, you can:

- See all your tasks listed clearly
- Find buttons to perform actions like adding new tasks

How the Main Window is Built

When you launch the To-Do App, here's a simplified sequence of events: The User starts the app, which triggers the Application Bootstrap. The Main Window is created and told to set up the action bar at the top and the task list area. The Main Window puts these components inside itself and becomes ready. Finally, the app shows the Main Window to the user.

The Code

The **MainWindow.java** file, located at **src/main/java/todo/com/ui/MainWindow.java**, is responsible for building our main application window. The MainWindow class extends JFrame, which is a special

class from Java's Swing library used for creating graphical user interfaces. When `MainWindow` extends `JFrame`, it means `MainWindow` IS a `JFrame`, making it an actual window that can appear on your screen.

Key Setup Instructions in the Constructor:

- **`setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)`**: Closes the window and stops the application when you click the 'X' button
- **`setSize(1200, 600)`**: Sets the initial window size to 1200 pixels wide and 600 pixels tall
- **`setTitle()`**: Sets the text that appears at the top of the window

The `MainWindow` uses a `BorderLayout` to organize its components. `BorderLayout` divides the panel into five regions: North, South, East, West, and Center. The `TopBarPanel` is placed at the NORTH (top), and the `TaskTablePanel` is placed at the CENTER, where it takes up all remaining space. This is perfect for the task list display.

Chapter 2: Task List Display

Understanding the Task List Display

Imagine you have a personal assistant who writes down all your tasks on a neat whiteboard. That's essentially what the Task List Display does for our To-Do App! It's the dedicated area where all your to-do items are shown in an organized, easy-to-read way.

What is the Task List Display?

The main problem this component solves is **seeing your progress and what's next**. Without it, your tasks would be hidden, and you wouldn't know what you need to do. The Task List Display brings your tasks out into the open, allowing you to quickly check details, see their status (done or not done), and even deadlines. It's your visual tracker for everything you need to accomplish!

What You See and Do

The Task List Display is the large central area where you'll find your list of tasks. In this area, you expect to:

- See each task clearly listed
- Notice if a task is "done" or "pending"
- See other details like deadlines
- Experience visual feedback when you move your mouse over a task (like highlighting)

Making Tasks Look Good: RowStyle

The **RowStyle.java** file defines how each task row will look. It's responsible for:

- **Alternating row colors**: Different background colors for odd and even rows (like a striped notebook)
- **Hover and selection feedback**: Highlighting when your mouse is over it or when selected
- **Borders and spacing**: Adding lines between tasks to keep them separate and neat

The RowStyle class uses several color constants:

- **COLOR_HOVER_SELECT**: Light gray color when mouse is over or row is selected
- **COLOR_STRIPE**: Very light gray for striped rows
- **ROW_BOTTOM_BORDER**: A thin dark gray line at the bottom of each row

The `apply()` method checks if the current row should be highlighted. If it's selected or your mouse is hovering over it, it uses the hover color. Otherwise, it creates a striped effect by alternating between

white and light gray backgrounds, making the list easier to scan.

Chapter 3: Task Action Bar

Your Control Panel for Task Management

Imagine you have a remote control for your To-Do App. The **Task Action Bar** is exactly that! It's the dedicated area, usually at the top of the app, where you find all the buttons and controls to perform actions related to your tasks.

What is the Task Action Bar?

The main problem this component solves is **task management efficiency**. Without it, you'd have no easy way to tell the app what you want to do (add, search, filter, etc.). The Task Action Bar puts all these important controls right at your fingertips.

What You See and Do

When you look at the Task Action Bar, you'll see a collection of buttons and possibly other input fields. For example:

- **Add New Task:** A button you click to create a brand new task
- **Filter:** Buttons or dropdowns to narrow down your task list
- **Sort:** Options to arrange your tasks by due date, priority, etc.

The TopBarPanel Code

The **TopBarPanel.java** file, located at **src/main/java/todo/com/ui/TopBarPanel.java**, creates the action bar. **TopBarPanel** extends **JPanel**, making it a rectangular container within the main window. It uses **FlowLayout(FlowLayout.LEFT, 10, 10)** to arrange components one after another, like words in a sentence, starting from the left with 10 pixels of spacing.

The key feature is the **setAddTaskListener()** method, which allows other parts of the app to register what should happen when the 'Add New Task' button is clicked. The **MainWindow** uses this method to provide the specific action: open the **AddTaskDialog** (the form for adding tasks).

This modular design makes the app flexible: the **TopBarPanel** creates the button, and **MainWindow** defines what happens when it's clicked. This separation of concerns keeps code organized and easy to maintain.

Chapter 4: Database Manager

The Unsung Hero: Persistent Memory

You click 'Add New Task' and enter a task. But what happens to that task when you close the app? Does it disappear? Definitely not! This is where the **Database Manager** comes in. It's the unsung hero that ensures your tasks are remembered, even after you close the application.

What is the Database Manager?

Imagine our To-Do App is like a bustling library, and you're constantly checking out books (tasks), returning them, or adding new ones. The **Database Manager** is the super-organized librarian for this library!

Its Job:

- **Store** new tasks you add
- **Retrieve** all existing tasks so they can be shown
- **Update** tasks (like marking them as "done")
- **Delete** tasks you no longer need

The main problem this component solves is **data persistence**. Without a Database Manager, any task you add would be lost forever as soon as you close the app. The Database Manager makes sure your to-do list is stored safely and consistently.

CRUD Operations

The Database Manager is responsible for fundamental operations on your tasks, summarized by the acronym **CRUD**:

- **Create**: Adding a brand new task
- **Read**: Getting all tasks or a specific task to display
- **Update**: Modifying an existing task
- **Delete**: Removing a task permanently

MongoDBManager Class

The **MongoDBManager.java** file is located at **src/main/java/todo/com/database/MongoDBManager.java**. Other parts of the app don't directly talk to MongoDB; instead, they send requests to the MongoDBManager.

Key Methods:

- **saveTask(String taskDescription)**: Saves a new task and returns a confirmation message
- **getAllTasks()**: Retrieves all tasks from the database
- **updateTask()**: Modifies an existing task
- **deleteTask()**: Removes a task permanently

The constructor initializes the connection to MongoDB. This happens when the MongoDBManager is first created, typically during the application startup process. By having clear methods like `saveTask` and `getAllTasks`, the `MongoDBManager` provides a simple interface for the rest of the app, abstracting away complex database operations.

Chapter 5: Application Bootstrap

The Starting Point: Bringing It All Together

We've journeyed through all the individual parts of our To-Do App: the Main Application Window, Task List Display, Task Action Bar, and Database Manager. But how do all these pieces start working together? This is where **Application Bootstrap** comes in.

What is Application Bootstrap?

Think of Application Bootstrap as the **"On" switch** for our entire To-Do App. It's the very first part of the code that runs when you launch the program. Its main job is to set up all the foundational elements of the application: initializing the Database Manager, creating the Main Application Window, and then making that window visible to you.

The Launch Sequence

When you launch the To-Do App, here's what happens in order:

1. **Initialize Database Manager:** The first step is to create the Database Manager. This sets up the connection to MongoDB so the app can store and retrieve tasks.
2. **Create Main Window:** Next, the Main Window is created. When this happens, the Main Window's constructor runs, which in turn sets up the Task List Display and Task Action Bar inside itself.
3. **Make Main Window Visible:** Finally, the Main Window is made visible to the user. At this point, the entire To-Do App is ready for use!

The App.java File

The heart of Application Bootstrap is the `main()` method in **`src/main/java/todo/com/App.java`**. This is the special method that Java looks for when you run a program – it's the official entry point.

The Bootstrap Process:

- **Import Necessary Classes:** The file imports `MainWindow`, `MongoDBManager`, and `SwingUtilities`
- **Initialize MongoDBManager:** A new `MongoDBManager` instance is created, setting up the database

connection

- **Use `SwingUtilities.invokeLater()`:** For Java Swing applications, all GUI updates must happen on a dedicated thread called the Event Dispatch Thread (EDT). This special rule prevents threading issues and ensures smooth UI performance
- **Create `MainWindow`:** Inside `invokeLater`, a new `MainWindow` is created
- **Set Visible:** Finally, `mainWindow.setVisible(true)` makes the entire application appear on screen

Why `SwingUtilities.invokeLater()`?

`SwingUtilities.invokeLater()` is a standard practice in Swing applications. It ensures that all GUI-related tasks (creating windows, updating displays, etc.) run on the Event Dispatch Thread. This prevents potential issues like thread conflicts and ensures that your UI updates smoothly and responsively.

Putting It All Together

When you run the `App` class, the `main()` method acts as our Application Bootstrap. It's like the opening sequence of a movie: it sets the stage, introduces the key players (Database Manager and Main Window), and then curtain up – the show begins! The user sees the fully functional To-Do App ready to accept tasks.

Architecture Overview

How All Components Work Together

Now that we've explored each individual component, let's understand how they all work together to create a complete, functional To-Do application.

The Component Relationships:

Application Bootstrap (App.java)

↓ Initializes

Database Manager (MongoDBManager.java) + Main Window (MainWindow.java)

↓ MainWindow contains and orchestrates

Task List Display (TaskTablePanel.java) + Task Action Bar (TopBarPanel.java)

↓ Both interact with

Database Manager (for saving/retrieving tasks)

Data Flow Example: Adding a New Task

1. **User Action:** User clicks the 'Add New Task' button in the Task Action Bar
2. **Dialog Opens:** TopBarPanel's addTaskListener triggers, opening the AddTaskDialog form
3. **User Inputs Data:** User types task details into the form and clicks 'Save'
4. **Database Save:** AddTaskDialog calls MongoDBManager.saveTask() to store the task
5. **Database Confirms:** MongoDBManager saves to MongoDB and returns confirmation
6. **Refresh Display:** TaskTablePanel retrieves updated task list from MongoDBManager
7. **User Sees Update:** The new task appears in the Task List Display with proper styling from RowStyle

Key Design Principles

- **Separation of Concerns:** Each component has a specific responsibility. UI components don't directly access the database; they go through the Database Manager.

- **Modularity:** Each component can be understood and modified independently.
- **Reusability:** Components like RowStyle can be applied across different parts of the UI.
- **Event-Driven:** User actions trigger listeners, which then request the necessary operations.
- **Abstraction:** The Database Manager abstracts away complex MongoDB operations behind simple methods.

Conclusion

You've now completed a comprehensive journey through the To-Do App's architecture! From understanding the Main Application Window that holds everything, to the Task List Display where tasks are visualized with beautiful styling, the Task Action Bar that provides intuitive controls, the Database Manager that ensures your data persists, and finally the Application Bootstrap that brings it all to life – each component plays a vital role.

The beauty of this architecture is in its clarity and organization. Each component knows its role and communicates with other components through well-defined interfaces. This makes the application maintainable, extensible, and easy to understand for future developers.

As you work with this To-Do App, remember that:

- The **MainWindow** is your canvas – everything else is painted on it
- The **TaskTablePanel** is your window to all your tasks
- The **TopBarPanel** is your control center for managing tasks
- The **MongoDBManager** is your persistent memory
- The **App.java** is the magic wand that makes it all happen

Whether you're using this To-Do App to manage your daily tasks or learning from its architecture to build your own applications, understand that this is just the beginning. The principles and patterns used here – separation of concerns, event-driven architecture, persistent data management, and clean UI design – are foundational to building robust, scalable applications.

Happy task managing, and happy coding!